02. Variables and Data Types

1 What is a Variable ?

- A variable is a name given to a value so that we can use it later in the program.
- We can store different types of data in a variable.
- Variable names can contain letters, numbers and underscore (_).
- Variable names cannot start with a number.

2 Rules for Variable Names

- Must start with a letter or underscore (_).
- Can contain letters, numbers and underscore (_).
- Case-sensitive (name, Name and NAME are different).
- Cannot use Python reserved keywords (like if, else, for, etc.).
- Should be meaningful and descriptive.

3 Creating Variables

```
# Example
name = "Rahul"
age = 28
salary = 75000.50
is-employee = True
```

Output

```
Rahul
28
75000.50
True
```

4 Assigning Multiple Values

```
# Multiple variables
a, b, c = 10, 20, 30
print(a, b, c)
```

Output

```
10 20 30
```

```
# Same value to multiple variables
x = y = z = 5
print(x, y, z)
```

Output

```
5 5 5
```

5 Data Types in Python

Python has different built-in data types to store different kinds of values.

Data Type	Description
int	Stores whole numbers (e.g., 10, -5)
float	Stores decimal numbers (e.g., 3.14)
str	Stores text (e.g., "Rahul")
bool	Stores True or False
list	Stores multiple values in a list
tuple	Stores multiple values (immutable)
set	Stores unique values
dict	Stores key-value pairs

6 Examples of Data Types

```
# Examples
a = 10                          # int
b = 3.14                        # float
c = "Python"                    # str
d = True                        # bool
e = [1, 2, 3]                   # list
f = (4, 5, 6)                   # tuple
g = {1, 2, 3}                   # set
h = {"name":"Rahul","age":28} # dict

print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
print(type(f))
print(type(g))
print(type(h))
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>
<class 'tuple'>
<class 'set'>
<class 'dict'>
```

7 Type Conversion

- We can convert one data type to another using built-in functions.
- Common functions: int(), float(), str(), bool()

```
# Type conversion examples
a = "100"
b = int(a)
c = float(a)
d = str(50)
print(type(b), b)
print(type(c), c)
print(type(d), d)
```

Output

```
<class 'int'> 100
<class 'float'> 100.0
<class 'str'> 50
```

8 Key Takeaways

- Variables are used to store values.
- Python supports multiple data types.
- Follow proper naming rules for variables.
- Use meaningful names.
- We can convert between data types using built-in functions.
- Understanding data types is important for writing correct programs.

Rahul Yadav

03. Operators in Python

1 What are Operators ?

- Operators are special symbols used to perform operations on values and variables.
- They help us perform arithmetic, comparison, logical and other operations.
- Operators take one or more operands (values or variables) and return a result.
- Python supports different types of operators.

2 Types of Operators

1. Arithmetic Operators (+, -, *, /, //, %, **)
2. Comparison Operators (==, !=, >, <, >=, <=)
3. Logical Operators (and, or, not)
4. Assignment Operators (=, +=, -=, *=, /=, etc.)
5. Bitwise Operators (&, |, ^, ~, <<, >>)
6. Membership Operators (in, not in)
7. Identity Operators (is, is not)

3 Arithmetic Operators

Used to perform mathematical operations.

Operator	Description	Example	Output
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 3	3.3333
//	Floor Division	10 // 3	3
%	Modulus	10 % 3	1
**	Exponentiation	2 ** 3	8

4 Comparison Operators

Used to compare two values. It returns True or False.

Operator	Description	Example	Output
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater than or equal	5 >= 5	True
<=	Less than or equal	3 <= 5	True

5 Logical Operators

Used to combine multiple conditions.

Operator	Description	Example	Output
and	True if both are True	True and False	False
or	True if any one is True	True or False	True
not	Reverses the result	not True	False

6 Assignment Operators

Used to assign values to variables.

Operator	Example	Same as
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
//=	x //= 3	x = x // 3
%=	x %= 3	x = x % 3
**=	x **= 3	x = x ** 3

7 Bitwise Operators

- Used to perform operations on binary numbers (at bit level).

Operator	Description	Example	Output
&	Bitwise AND	5 & 3	1
|	Bitwise OR	5 | 3	7
^	Bitwise XOR	5 ^ 3	6
~	Bitwise NOT	~5	-6
<<	Left Shift	5 << 1	10
>>	Right Shift	5 >> 1	2

8 Membership and Identity Operators

Membership Operators (in, not in)

Operator	Example	Output
in	3 in [1, 2, 3]	True
not in	4 not in [1, 2, 3]	True

Identity Operators (is, is not)

Operator	Example	Output
is	a is b	False
is not	a is not b	True

9 Key Takeaways

- Operators are used to perform operations on values and variables.
- Python supports arithmetic, comparison, logical, assignment, bitwise, membership and identity operators.
- Each operator has a specific purpose and returns a result.
- Using the right operator makes your code more readable and efficient.
- Practice with examples to understand how each operator works.

Rahul Yadav

04. Conditional Statements in Python

1 What are Conditional Statements ?

- Conditional statements are used to execute different blocks of code based on conditions.
- They help the program make decisions.
- Python supports if, elif and else statements.
- Conditions always evaluate to True or False.

2 Comparison Operators (used in conditions)

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

3 if Statement

- Executes a block of code if the condition is True.
- It is the simplest form of conditional statement.

```
# Example
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

Output

```
You are eligible to vote.
```

4 if - else Statement

- Executes one block if the condition is True and another block if the condition is False.

```
# Example
age = 16
if age >= 18:
    print("You can vote.")
else:
    print("You cannot vote.")
```

Output

```
You cannot vote.
```

5 if - elif - else Statement

- Used when we have multiple conditions to check.
- The first True condition block will be executed.

```
# Example
marks = 75
if marks >= 90:
    print("Grade: A")
elif marks >= 70:
    print("Grade: B")
elif marks >= 50:
    print("Grade: C")
else:
    print("Grade: D")
```

Output

```
Grade: B
```

6 Nested if Statement

- An if statement inside another if statement is called a nested if.
- Useful when we have multiple conditions depending on earlier conditions.

```
# Example
age = 20
is_student = True
if age >= 18:
    if is_student:
        print("Adult Student")
    else:
        print("Adult")
else:
    print("Minor")
```

Output

```
Adult Student
```

7 Short-hand if Statement

- Used for simple conditions in a single line.

```
# Example
age = 20
result = "Adult" if age >= 18 else "Minor"
print(result)
```

Output

```
Adult
```

8 Key Takeaways

- Conditional statements help in decision making.
- Use if for a single condition.
- Use if-else when there are two possibilities.
- Use if-elif-else for multiple conditions.
- Python also supports nested if and short-hand if for concise code.
- Conditions always return True or False.

Rahul Yadav

05. Loops in Python

1 What are Loops ?

- Loops are used to execute a block of code multiple times.
- They help us avoid writing the same code again and again.
- Python supports two main types of loops: for loop and while loop.
- Loops are useful when we work with collections (list, tuple, string, etc.) or when we need to repeat a task for a certain number of times.

2 Types of Loops

- for loop - Used to iterate over a sequence (list, tuple, string, range, etc.).
- while loop - Used to execute a block of code as long as a condition is True.
- nested loops - A loop inside another loop.
- Loop control statements - break, continue, pass.

3 for Loop with Range

- The range() function generates a sequence of numbers.
- Syntax: range(start, stop, step)

```
# Example
for i in range(5):
    print(i)
```

Output

```
0
1
2
3
4
```

4 for Loop with List

- We can iterate directly over a list, tuple, string or any iterable object.
- Each item will be assigned to the loop variable.

```
# Example
fruits = ["Apple", "Banana", "Mango"]
for fruit in fruits:
    print(fruit)
```

Output

```
Apple
Banana
Mango
```

5 while Loop

- The while loop runs as long as the condition is True.
- We must update the condition inside the loop to avoid an infinite loop.

```
# Example
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

Output

```
1
2
3
4
5
```

6 Loop Control Statements

- break - Terminates the loop.
- continue - Skips the current iteration.
- pass - Does nothing (used as a placeholder).

```
# break example
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output

```
0
1
2
3
4
```

```
# continue example
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Output

```
0
1
3
4
```

7 Nested Loops

- A loop inside another loop is called nested loop.
- Useful for working with multi-dimensional data like matrix.

```
# Example
for i in range(3):
    for j in range(3):
        print(i, j)
```

Output

```
0 0
0 1
0 2
1 0
1 1
1 2
2 0
2 1
2 2
```

8 Key Takeaways

- Loops help us execute code multiple times.
- for loop is used for iterating over sequences.
- while loop runs until a condition becomes False.
- Use break, continue and pass to control the flow.
- Nested loops are useful for multi-dimensional data.
- Loops make the code shorter, cleaner and efficient.

Rahul Yadav

06. Functions in Python

1 What are Functions ?

- Functions are reusable blocks of code used to perform a specific task.
- They help us avoid code repetition.
- A function can take input (parameters) return output and can be used many times in a program.
- Python provides many built-in functions and we can also create our own functions (user-defined functions).

2 Advantages of Functions

- Code reusability
- Makes code modular and organized
- Easier to read and maintain
- Reduces code duplication
- Helps in dividing a complex problem into smaller parts
- Supports parameters and return values

3 Defining and Calling a Function

- We use the def keyword to define a function.
- A function is called by its name followed by ().

```
# Example
def greet():
    print("Hello, Rahul!")

greet()   # calling the function
```

Output

```
Hello, Rahul!
```

4 Function with Parameters

- We can pass values to a function using parameters.
- Parameters allow the function to work with different input values.

```
# Example
def add(a, b):
    return a + b
result = add(5, 3)
print(result)
```

Output

```
8
```

5 Function with Return Value

- A function can return a value using the return keyword.
- The returned value can be stored in a variable and used later.

```
# Example
def multiply(a, b):
    return a * b
result = multiply(4, 6)
print(result)
```

Output

```
24
```

6 Default Parameters

- We can provide default values to parameters.
- If no value is passed, the default value is used.

```
# Example
def greet(name, msg = "Hello"):
    print(f"{msg}, {name}!")

greet("Rahul")
greet("Preeti", "Hi")
```

Output

```
Hello, Rahul!
Hi, Preeti!
```

7 Variable Length Arguments

- *args allows us to pass any number of positional arguments (stored as a tuple).
- **kwargs allows us to pass any number of keyword arguments (stored as a dictionary).

```
# Example: *args
def add_all(*args):
    return sum(args)

print(add_all(1, 2, 3))
print(add_all(10, 20, 30, 40))
```

Output

```
6
100
```

```
# Example: **kwargs
def show_info(**kwargs):
    for key, value in kwargs.items():
        print(key, "=", value)

show_info(name="Rahul",
          age=28, city="Jaipur")
```

Output

```
name = Rahul
age = 28
city = Jaipur
```

8 Scope of Variables in Functions

- Variables inside a function are local to that function.
- Variables outside a function are global variables.
- Local and global variables can have the same name but they are different.

```
# Example
x = 10    # global variable
def show():
    x = 20   # local variable
    print("Inside function:", x)
show()
print("Outside function:", x)
```

Output

```
Inside function: 20
Outside function: 10
```

9 Key Takeaways

- Functions help in code reusability and modularity.
- Use def to define a function.
- Functions can have parameters and return values.
- Use default parameters, *args and **kwargs when needed.
- Understand the scope of variables (local vs global).
- Functions make the code clean, readable and maintainable.

Rahul Yadav

07. Strings in Python

1 What is a String ?

- A string is a sequence of characters enclosed within single quotes (' '), double quotes (" ") or triple quotes (''' ''').
- Strings are used to store text data.
- Strings are immutable (cannot be changed).
- Each character in a string has an index starting from 0.

2 Creating Strings

- We can create a string using single quotes, double quotes or triple quotes.
- Triple quotes are used for multi-line strings.
- We can also create strings using the str() function.

```
# Example
name = 'Rahul'
msg = "Hello Python"
text = '''This is
    a multi-line
    string'''
num_str = str(100)
```

```
Output
Rahul
Hello Python
This is
a multi-line
string
100
```

3 String Indexing and Slicing

- Each character in a string has an index.
- We can access characters using index.
- We can also get a substring using slicing.

```
# Example
name = "Python"
print(name[0])   # first character
print(name[3])   # fourth character
print(name[-1])  # last character
print(name[0:3]) # substring (0 to 2)
print(name[2:])  # from index 2 to end
```

```
Output
P
h
n
Pyt
thon
```

4 Common String Operations

- We can concatenate strings using + operator.
- We can repeat a string using * operator.
- We can check if a substring exists using in.
- We can find the length of a string using len().

```
# Example
a = "Hello"
b = "Python"
print(a + " " + b)
print(a * 3)
print("Py" in b)
print(len(b))
```

```
Output

Hello Python
HelloHelloHello
True
6
```

5 Useful String Methods (Part 1)

- Python provides many built-in methods to work with strings.
- These methods help in common string operations like changing case, removing spaces, etc.

Method	Description	Example	Output
upper()	Converts to uppercase	"python".upper()	PYTHON
lower()	Converts to lowercase	"PYTHON".lower()	python
strip()	Removes spaces	" hi ".strip()	hi
replace()	Replaces substring	"a-b-c".replace("-",",")	a,b,c
split()	Splits into list	"a b c".split()	['a','b','c']

6 Useful String Methods (Part 2)

Method	Description	Example	Output
find()	Returns index	"hello".find("e")	1
count()	Counts substring	"banana".count("a")	3
startswith()	Checks prefix	"python".startswith("py")	True
endswith()	Checks suffix	"python".endswith("on")	True
title()	Title case	"hello world".title()	Hello World
capitalize()	First letter upper	"python".capitalize()	Python
isalpha()	Check letters only	"abc".isalpha()	True
isdigit()	Check digits only	"123".isdigit()	True
isalnum()	Letters and digits	"abc123".isalnum()	True

7 String Formatting

- We can insert values into strings using different methods.
- Common ways: f-strings, format() and % operator.

```
# Example
name = "Rahul"
age = 28
# f-string
print(f"My name is {name} and age is {age}")
# format()
print("My name is {} and age is {}".format(name, age))
# % operator
print("My name is %s and age is %d" % (name, age))
```

```
Output
My name is Rahul
and age is 28

My name is Rahul
and age is 28

My name is Rahul
and age is 28
```

8 Key Takeaways

- A string is a sequence of characters used to store text data.
- Strings are immutable.
- We can access characters using index and slicing.
- Python provides many useful string methods.
- We can concatenate, repeat and find substrings in strings.
- String formatting helps in creating dynamic strings.
- Understanding string operations is important for real-world applications like file handling, data processing and web development.

Rahul Yadav

08. Lists in Python

1 What is a List ?

- A list is a collection of items stored in a single variable.
- Lists can hold multiple values of different data types (int, float, string, etc.).
- Lists are ordered, mutable (can be changed) and allow duplicate values.
- Lists are created using square brackets [ ].

2 Creating a List

- We can create a list with values separated by commas.
- A list can contain values of different data types.
- A list can also be created using the list() constructor.

# Example	Output
numbers = [1, 2, 3, 4]	[1, 2, 3, 4]
fruits = ["Apple", "Banana", "Mango"]	['Apple', 'Banana', 'Mango']
mixed = [1, "Rahul", 3.5, True]	[1, 'Rahul', 3.5, True]
empty_list = []	[]
list() constructor: list([10, 20, 30])	[10, 20, 30]

3 Accessing Elements

- List elements can be accessed using index.
- Index starts from 0.
- We can also use negative indexing to access elements from the end.

# Example	Output
fruits = ["Apple", "Banana", "Mango"]	
print(fruits[0])	Apple
print(fruits[1])	Banana
print(fruits[2])	Mango
print(fruits[-1]) # last element	Mango
print(fruits[-2]) # second last	Banana

4 List Slicing

- We can get a part of the list using slicing.
- Syntax: list[start : stop : step]
- start index is included, stop index is excluded.

# Example	Output
nums = [10, 20, 30, 40, 50]	
print(nums[1:4])	[20, 30, 40]
print(nums[:3]) # from start	[10, 20, 30]
print(nums[2:]) # till end	[30, 40, 50]
print(nums[::2]) # step by 2	[10, 30, 50]
print(nums[::-1]) # reverse	[50, 40, 30, 20, 10]

5 Modifying Lists

- Lists are mutable, so we can change, add, remove or update elements.

# Example	Output
nums = [10, 20, 30]	
nums[1] = 50 # update value	[10, 50, 30]
nums.append(40) # add at end	[10, 50, 30, 40]
nums.insert(1, 25) # add at index	[10, 25, 50, 30, 40]
nums.remove(30) # remove value	[10, 25, 50, 40]
nums.pop() # remove last	[10, 25, 50]
nums.pop(1) # remove at index	[10, 50]
nums.clear() # remove all	[]

6 List Methods

- Python provides many built-in methods for lists.

Method	Description	Example
append(x)	Add element at end	nums.append(10)
insert(i, x)	Add at specific index	nums.insert(1, 10)
extend(iterable)	Add multiple elements	nums.extend([1,2,3])
remove(x)	Remove first occurrence	nums.remove(20)
pop(i)	Remove and return element	nums.pop(0)
clear()	Remove all elements	nums.clear()
index(x)	Return index of element	nums.index(30)
count(x)	Count occurrences	nums.count(10)
sort()	Sort list (ascending)	nums.sort()
reverse()	Reverse the list	nums.reverse()
copy()	Return a shallow copy	new_list = nums.copy()

7 Iterating through a List

- We can use a for loop to iterate over list elements.

# Example	Output
fruits = ["Apple", "Banana", "Mango"] for fruit in fruits: print(fruit)	Apple Banana Mango

8 Key Takeaways

- A list is an ordered, mutable collection of items.
- Lists can store different data types.
- We can access elements using index and slicing.
- Lists provide many useful methods for modification.
- Lists allow duplicate values.
- Lists are widely used in real-world applications like data processing, storage and manipulation.

Rahul Yadav

09. Tuples in Python

1 What is a Tuple ?

- A tuple is an ordered, immutable collection of items.
- Tuples can hold multiple values of different data types (int, float, string, etc.).
- Tuples are created using round brackets ().
- Since tuples are immutable, we cannot change, add or remove elements.
- Tuples are useful when we want to store fixed data that should not be modified.

2 Creating a Tuple

- We can create a tuple by enclosing values in parentheses ().
- A tuple can contain values of different data types.
- A tuple with a single element must have a comma (,) after the element.

# Example	Output
t1 = (1, 2, 3)	(1, 2, 3)
t2 = ("Rahul", 28, 5.10)	("Rahul", 28, 5.1)
t3 = (1,) # single element	(1,)
t4 = tuple([10, 20, 30])	(10, 20, 30)
t5 = () # empty tuple	()

3 Accessing Tuple Elements

- We can access elements using index (starts from 0).
- Negative indexing can be used to access from the end.

# Example	Output
t = (10, 20, 30, 40, 50)	
print(t[0]) # first element	10
print(t[3]) # fourth element	40
print(t[-1]) # last element	50
print(t[-2]) # second last	40
print(t[1:4]) # slicing	(20, 30, 40)

4 Tuple Operations

- We can perform common operations like concatenation (+), repetition (*), checking membership (in), and finding length (len().

# Example	Output
t1 = (1, 2, 3)	
t2 = (4, 5)	
print(t1 + t2) # concatenation	(1, 2, 3, 4, 5)
print(t1 * 3) # repetition	(1, 2, 3, 1, 2, 3, 1, 2, 3)
print(2 in t1) # membership	True
print(len(t1)) # length	3

5 Tuple Methods

- Tuples have limited methods because they are immutable.
- Mainly, we use count() and index().

Method	Description	Example	Output
count(x)	Returns number of occurrences	t = (1, 2, 2, 3) print(t.count(2))	2
index(x)	Returns index of first occurrence	t = (10, 20, 30) print(t.index(20))	1

6 Tuple vs List

- Both are ordered collections.
- List is mutable (can be changed).
- Tuple is immutable (cannot be changed).
- List uses [] and Tuple uses ().

Feature	List	Tuple
Syntax	[1, 2, 3]	(1, 2, 3)
Mutable	Yes	No
Methods	Many methods	Only count(), index()
Performance	Slower	Faster
Use Case	Dynamic data	Fixed/constant data

7 When to Use Tuples ?

- When we have fixed data that should not be changed.
- To store multiple values and return from a function.
- When we want to use a collection as a dictionary key (since tuples are hashable).
- To improve performance (tuples are faster than lists).

# Example	Output
coordinates = (28.7041, 77.1025) print(coordinates)	(28.7041, 77.1025)
person = ("Rahul", 28, "Developer") print(person)	('Rahul', 28, 'Developer')

8 Key Takeaways

- A tuple is an ordered, immutable collection.
- Created using ().
- Supports different data types.
- Access elements using index and slicing.
- Common operations: +, *, in, len().
- Main methods: count() and index().
- Use tuples when data should not be changed.
- Tuples are useful for fixed data, function return values and as dictionary keys.
- They are faster and more memory efficient than lists.

Rahul Yadav

10. Dictionaries in Python

1 What is a Dictionary?

- A dictionary is a collection of key-value pairs.
- It is unordered (before Python 3.7) and ordered (from Python 3.7+).
- Keys are unique and immutable (string, number, or tuple).
- Values can be of any data type and can be changed (mutable).
- Dictionaries are created using curly braces { } and use key: value pairs.

2 Creating a Dictionary

- We can create a dictionary using { } with key: value pairs.
- Keys must be unique.
- Values can be of any data type.
- We can also create a dictionary using the dict() constructor.

# Example	Output
`person = {"name": "Rahul", "age": 28, "city": "Jaipur"}` `print(person)`	`{'name': 'Rahul', 'age': 28, 'city': 'Jaipur'}`
`emp = dict(id=101, name="Preeti")` `print(emp)`	`{'id': 101, 'name': 'Preeti'}`

3 Accessing Dictionary Elements

- We can access values using the key inside square brackets [_].
- We can also use the get() method to avoid errors if a key does not exist.

# Example	Output
`person = {"name": "Rahul", "age": 28}`	
`print(person["name"])`	Rahul
`print(person.get("age"))`	28
`print(person.get("city", "Not Found"))`	Not Found

4 Adding and Updating Elements

- We can add a new key-value pair using assignment.
- If the key already exists, the value will be updated.

# Example	Output

`person = {"name": "Rahul", "age": 28}`
`person["city"] = "Jaipur"  # add new key`
`print(person)`
	`{'name': 'Rahul', 'age': 28, 'city': 'Jaipur'}`

`person["age"] = 29  # update value` `print(person)`	`{'name': 'Rahul', 'age': 29, 'city': 'Jaipur'}`

5 Removing Elements

- We can remove a key-value pair using pop(), popitem() or del.
- pop() removes a specific key and returns its value.
- popitem() removes the last inserted key-value pair.
- del removes the key completely.

# Example	Output

`person = {"name": "Rahul", "age": 28, "city": "Jaipur"}`
`removed = person.pop("age")`
`print(removed)`
	28

`print(person)`	`{'name': 'Rahul', 'city': "Jaipur"}`
`person.popitem()  # removes last item` `print(person)`	`{'name': 'Rahul"}`
`del person["name"]` `print(person)`	{ }

6 Dictionary Methods

- Python provides many useful built-in methods for dictionaries.
- These methods help in working with keys, values and items.

Method	Description	Example
keys()	Returns all keys	d.keys()
values()	Returns all values	d.values()
items()	Returns key-value pairs	d.items()
get(key)	Returns value for a key	d.get("name")
pop(key)	Removes key and returns value	d.pop("age")
popitem()	Removes last key-value pair	d.popitem()
update()	Updates dictionary with another	d.update({"city":"Jaipur"})
clear()	Removes all items	d.clear()
copy()	Returns a shallow copy	d.copy()

7 Iterating through a Dictionary

- We can use for loop to iterate over keys, values or key-value pairs.

# Example	Output

`person = {"name": "Rahul", "age": 28, "city": "Jaipur"}`
`# iterate over keys`
`for key in person:`
`  print(key)`
	name
	age
	city

`# iterate over values`
`for value in person.values():`
`  print(value)`
	Rahul
	28
	Jaipur

`# iterate over key-value pairs`
`for k, v in person.items():`
`  print(k, "=", v)`
	name = Rahul
	age = 28
	city = Jaipur

8 Key Takeaways

- A dictionary is a collection of key-value pairs.
- Keys are unique and immutable.
- Values can be of any data type.
- We can create, access, add, update and remove elements easily.
- Useful methods: keys(), values(), items(), get(), pop(), popitem(), update(), clear(), copy().
- Dictionaries are widely used in real-world applications like storing JSON data, configuration settings, database records, etc.
- They provide fast data access using keys.

Rahul Yadav

11. Conditional Statements in Python

1 What are Conditional Statements?

- Conditional statements allow us to execute different blocks of code based on certain conditions.
- They help in decision making.
- The condition is evaluated and if it is True, a specific block of code is executed.
- Python provides if, elif and else for conditional statements.
- Conditions usually use comparison operators and return True or False.

2 Syntax of if, elif, else

```
if condition1:
    # code block 1
elif condition2:
    # code block 2
else:
    # code block 3
```

- if is used to check the first condition.
- elif (else if) checks another condition if the previous one is False.
- else is executed if none of the conditions are True.
- Indentation is important in Python.

3 if Statement

- The if statement is used to execute a block of code when a condition is True.
- If the condition is False, the code block is skipped.

# Example	Output

`age = 18`
`if age >= 18:`
`  print("You are an adult")`
	You are an adult

4 if - else Statement

- The if-else statement executes one block if the condition is True and another block if it is False.

# Example	Output

`age = 16`
`if age >= 18:`
`  print("You are an adult")`
`else:`
`  print("You are a minor")`
	You are a minor

5 if - elif - else Statement

- We can use multiple conditions using elif.
- It is useful when we have more than two possible conditions.

# Example	Output

`marks = 76`
`if marks >= 90:`
`  print("Grade A")`
`elif marks >= 75:`
`  print("Grade B")`
`elif marks >= 60:`
`  print("Grade C")`
`else:`
`  print("Grade D")`
	Grade B

6 Nested if Statement

- We can use an if statement inside another if statement.
- It is useful when we need to check multiple conditions step by step.

# Example	Output

`age = 20`
`is_student = True`
`if age >= 18:`
`  if is_student:`
`  print("Adult Student")`
`  else:`
`  print("Adult but not Student")`
`else:`
`  print("Minor")`
	Adult Student

7 Conditional Statements with Logical Operators

- We can use logical operators (and, or, not) to combine multiple conditions.
- These help in writing more powerful conditions.

Operator	Description	Example	Result
and	True if both conditions are True	(5 > 3 and 10 > 5)	True
or	True if at least one condition is True	(5 > 3 or 2 > 10)	True
not	Inverts the result (True → False)	not(5 > 3)	False

# Example	Output

`age = 22`
`has_id = True`
`if age >= 18 and has_id:`
`  print("You can vote")`
`else:`
`  print("You cannot vote")`
	You can vote

8 Key Takeaways

- Conditional statements help in decision making.
- Python provides if, elif and else.
- Use comparison operators (>, <, >=, <=, ==, !=).
- Conditions return True or False.
- Use elif when you have multiple conditions.
- Use logical operators (and, or, not) to combine conditions.
- We can also use nested if for complex conditions.
- Indentation is important in Python.
- Conditional statements are widely used in real applications like form validation, authentication, filtering data, etc.

Rahul Yadav

12. Functions in Python

1 What is a Function ?

- A function is a reusable block of code that performs a specific task.
- Functions help in organizing code, reducing duplication and improving readability.
- We can define a function using the **def** keyword.
- Functions can take input (parameters) and can return output (values).
- Functions are executed only when they are called.

2 Defining and Calling a Function

- We can define a function using **def** keyword, followed by function name, parentheses () and colon :.
- Code inside a function is written with indentation.
- We can call a function by using its name followed by parentheses ().

# Example	Output
def greet(): print("Hello, Python!") greet() # function call	Hello, Python!

3 Function with Parameters

- We can pass values to a function using parameters.
- Parameters are like variables that receive values when the function is called.

# Example	Output
def greet(name): print("Hello, " + name + "!") greet("Rahul") greet("Preeti")	Hello, Rahul! Hello, Preeti!

4 Function with Return Value

- A function can return a value using the return keyword.
- We can store the returned value in a variable and use it further in the program.

# Example	Output
def add(a, b): return a + b result = add(5, 3) print(result)	8

5 Default Parameters

- We can provide default values to parameters.
- If no value is passed, the default value is used.

# Example	Output
def greet(name = "Guest"): print("Hello, " + name + "!") greet() greet("Rahul")	Hello, Guest! Hello, Rahul!

6 Keyword Arguments

- We can pass arguments using parameter names in any order.
- It makes the code more readable.

# Example	Output
def student(name, age): print("Name:", name) print("Age:", age) student(age=28, name="Rahul")	Name: Rahul Age: 28

7 Arbitrary Arguments (*args)

- We can pass multiple arguments to a function using *args.
- *args collects all the arguments in a tuple.

# Example	Output

def total(*args):
  print(args) # args is a tuple
  print("Sum:", sum(args))
total(1, 2, 3, 4)
	(1, 2, 3, 4)
	Sum: 10

8 Arbitrary Keyword Arguments (**kwargs)

- We can pass multiple keyword arguments using **kwargs.
- **kwargs collects all arguments in a dictionary.

# Example	Output
def person(**kwargs): print(kwargs) print("Name:", kwargs["name"]) print("Age:", kwargs["age"]) person(name="Rahul", age=28, city="Jaipur")	{'name': 'Rahul', 'age': 28, 'city': 'Jaipur'} Name: Rahul Age: 28

9 Scope of Variables in Functions

- Variables inside a function are local to that function.
- Variables defined outside the function are global.
- We can use the global keyword to modify a global variable inside a function.

# Example	Output
x = 10 # global variable def show(): print("Inside function:", x) show() print("Outside function:", x)	Inside function: 10 Outside function: 10

10 Key Takeaways

- Functions help in code reusability and better organization.
- Use **def** to define a function.
- Functions can have parameters and return values.
- Default parameters provide default values.
- Use *args for multiple positional arguments.
- Use **kwargs for multiple keyword arguments.
- Functions have local and global scope.
- Functions make code easier to maintain and debug.
- They are widely used in real-world Python applications.

Rahul Yadav

13. Modules and Packages in Python

1 What is a Module ?

- A module is a file containing Python code (functions, variables, classes, etc.).
- It helps in organizing code into separate files.
- Modules make the code reusable.
- We can use a module in another Python file using the import keyword.
- Python provides many built-in modules like math, random, datetime, etc.

2 Creating a Module

- We can create a module by writing Python code in a .py file.
- Suppose we create a file named calculator.py:

```
# calculator.py
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b
```

3 Importing a Module

- We can import a module using the import keyword.
- We can access functions and variables using dot notation (module_name.function_name).

# Example	Output
import calculator	
print(calculator.add(5, 3))	8
print(calculator.multiply(4, 6))	24

4 Importing Specific Functions

- We can import only specific functions from a module using the from ... import statement.
- This helps to avoid using the module name again and again.

# Example	Output
from calculator import add, multiply	
print(add(10, 20))	30
print(multiply(5, 6))	30

5 Using Alias (as)

- We can use an alias to give a shorter name to a module or function.
- It makes the code shorter and easier to write.

# Example	Output
import calculator as calc	
print(calc.add(7, 3))	10
print(calc.multiply(2, 5))	10

6 Built-in Modules

- Python comes with many built-in modules that we can use directly.
- Some commonly used modules are:

Module	Description	Example
math	Mathemetical functions	math.sqrt(16)
random	Random numbers	random.randint(1, 10)
datetime	Date and time	datetime.datetime.now()
os	OS related functions	os.listdir()
sys	System specific params	sys.version
json	JSON data handling	json.dumps({"a": 1})

7 Creating a Package

- A package is a collection of related modules.
- A package is a folder containing multiple .py files and a special file named __init__.py.

```
# Example Package Structure
mypackage/
    ├── __init__.py
    ├── module1.py
    └── module2.py
```

8 Importing from a Package

- We can import modules from a package using dot notation.

# Example	Output
from mypackage import module1 print(module1.greet("Rahul"))	Hello, Rahul!
from mypackage.module2 import say_hello print(say_hello("Preeti"))	Hello, Preeti!

9 The __init__.py File

- The __init__.py file makes a directory a Python package.
- It can be empty or can contain initialization code.
- We can also define what should be imported when using from package import *.

```
# Example (__init__.py)
from .module1 import greet
from .module2 import say_hello
__all__ = ["greet", "say_hello"]
```

10 Key Takeaways

- A module is a single Python file containing code.
- A package is a folder containing multiple modules.
- Use import to include a module.
- Use from ... import to import specific functions.
- Use alias (as) to shorten module names.
- Python provides many useful built-in modules.
- Packages help in organizing large projects.
- The __init__.py file is required to create a package.
- Modules and packages make the code reusable, organized and maintainable.

Rahul Yadav

14. Python Data Structures (Lists, Tuples, Sets, Dictionaries)

1 Overview of Data Structures

- Data structures are used to store and organize data in an efficient way.
- Python provides built-in data structures like list, tuple, set and dictionary.
- Each data structure has its own properties and use cases.
- They help in handling collections of data easily and efficiently.

2 List in Python

- A list is an ordered, mutable collection of items.
- It can contain elements of different data types.
- We can add, remove, update and access elements using index.
- Lists are created using square brackets [ ].

# Example	Output

`nums = [1, 2, 3, 4, 5]`
`print(nums[0])  # first element`
`nums.append(6)  # add element`
`nums.remove(3)  # remove element`
`print(nums)`
	`1`
	`[1, 2, 4, 5, 6]`

3 Tuple in Python

- A tuple is an ordered, immutable collection of items.
- It can contain different data types.
- Tuples are created using round brackets ( ).
- Since tuples are immutable, we cannot change, add or remove elements.

# Example	Output

`t = (1, 2, 3, 4)`
`print(t[0])  # first element`
`print(t[-1])  # last element`
`print(len(t))  # length`
	`1`
	`4`
	`4`

4 Set in Python

- A set is an unordered collection of unique items.
- It does not allow duplicate values.
- Sets are mutable (we can add or remove items).
- Sets are created using curly braces { } or set().

# Example	Output

`s = {1, 2, 2, 3, 4}`
`print(s)  # duplicates removed`
`s.add(5)  # add element`
`s.remove(3)  # remove element`
`print(s)`
	`{1, 2, 3, 4}`
	`{1, 2, 4, 5}`

5 Dictionary in Python

- A dictionary is an unordered collection of key-value pairs.
- Keys are unique and immutable.
- Values can be of any data type.
- Dictionaries are created using curly braces { } with key: value pairs.

# Example	Output

`d = {"name": "Rahul", "age": 28}`
`print(d["name"])`
`d["city"] = "Jaipur"  # add new key`
`print(d)`
	`Rahul`
	`{'name': 'Rahul', 'age': 28, 'city': 'Jaipur'}`

6 Common Operations

Operation	List	Tuple	Set	Dictionary
Ordered	Yes	Yes	No	No
Mutable	Yes	No	Yes	Yes (values)
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Syntax	[ ]	( )	{ } / set()	{key: value}
Access by index	Yes	Yes	No	By key
Add element	append()	Not allowed	add()	d[key] = value
Remove element	remove()	Not allowed	remove()	pop(key)
Use case	General	Fixed data	Unique data	Key-value data

7 Iterating through Data Structures

- We can use for loop to iterate through lists, tuples, sets and dictionaries.
- For dictionaries, we can iterate through keys, values or key-value pairs.

# Example	Output

`fruits = ["Apple", "Banana", "Mango"]`
`for f in fruits:`
`  print(f)`
	`Apple`
	`Banana`
	`Mango`

`person = {"name": "Rahul", "age": 28}`
`for k, v in person.items():`
`  print(k, "-", v)`
	`name - Rahul`
	`age - 28`

8 Useful Methods

Data Structure	Common Methods
List	append(), extend(), insert(), remove(), pop(), sort(), reverse(), index(), count()
Tuple	count(), index()
Set	add(), remove(), discard(), union(), intersection(), difference(), clear()
Dictionary	keys(), values(), items(), get(), pop(), update(), clear()

9 Nested Data Structures

- We can have nested data structures like list inside a list or dictionary inside a list.
- This is useful to represent complex real-world data.

# Example	Output

`data = {`
`  "name": "Rahul",`
`  "skills": ["Python", "Java", "SQL"],`
`  "details": {"age": 28, "city": "Jaipur"}`
`}`
`print(data["skills"][0])`
	`Python`

10 Key Takeaways

- Lists are ordered and mutable.
- Tuples are ordered and immutable.
- Sets are unordered and contain unique values.
- Dictionaries store key-value pairs.
- Each data structure has different methods and use cases.
- We can iterate through all data structures using for loop.
- Choose the right data structure based on the requirement.
- They are fundamental and widely used in real-world Python applications.

Rahul Yadav

15. File Handling in Python

1 What is File Handling ?

- File handling means working with files on your computer (reading, writing, updating, etc.).
- We can perform operations like create, read, write, append and delete files using Python.
- Files are used to store data permanently.
- Python provides built-in functions like open(), read(), write(), close() to work with files.
- We can also use with statement for better resource management.

2 Opening a File

- We use the open() function to open a file.
- It takes two parameters: file name and mode.
- Common modes: 'r' (read), 'w' (write), 'a' (append), 'x' (create), 'b' (binary), 't' (text).
- It returns a file object which we can use to perform operations.

# Example	Output
f = open("data.txt", "r")	Opens file in read mode
f = open("data.txt", "w")	Opens file in write mode
f = open("data.txt", "a")	Opens file in append mode
f = open("data.txt", "x")	Creates a new file
f = open("data.txt", "r+")	Read and write mode

3 Writing to a File

- We can write data to a file using the write() method.
- If the file does not exist, it will be created (in 'w' mode).
- If the file already exists, the old data will be overwritten in 'w' mode.

# Example	Output
f = open("data.txt", "w") f.write("Hello, Python!\n") f.write("File handling is easy.\n") f.close()	Creates a file and writes data into it.

4 Reading from a File

- We can read data from a file using read(), readline() or readlines().
- If the file does not exist, it will raise an error in 'r' mode.

# Example	Output
f = open("data.txt", "r") content = f.read() print(content) f.close()	Hello, Python! File handling is easy.

5 Reading Line by Line

- We can use readline() to read one line at a time.
- We can use readlines() to read all lines into a list.
- We can also use a for loop to iterate over lines.

# Example	Output
f = open("data.txt", "r") for line in f: print(line.strip()) f.close()	Hello, Python! File handling is easy.

6 Appending to a File

- We can add new data to an existing file using 'a' mode.
- The existing data will not be removed.

# Example	Output
f = open("data.txt", "a") f.write("This is a new line.\n") f.close()	Adds a new line to the existing file.

7 Using with Statement

- The with statement automatically closes the file.
- It is the recommended way to work with files.
- It helps to avoid memory leaks and errors.

# Example	Output
with open("data.txt", "r") as f: content = f.read() print(content)	Reads the file and automatically closes it.

8 Working with Different File Modes

Mode	Description	Example
r	Read (default)	open("file.txt", "r")
w	Write (overwrite)	open("file.txt", "w")
a	Append (add)	open("file.txt", "a")
x	Create (new file)	open("file.txt", "x")
r+	Read and write	open("file.txt", "r+")
w+	Write and read	open("file.txt", "w+")
a+	Append and read	open("file.txt", "a+")
b	Binary mode	open("image.jpg", "rb")
t	Text mode (default)	open("file.txt", "rt")

9 Deleting a File

- We can delete a file using the os module.
- If the file does not exist, it will raise an error.

# Example	Output
import os os.remove("old_file.txt") print("File deleted!")	File deleted!

10 Key Takeaways

- File handling helps to store and manage data permanently.
- Use open() with appropriate mode.
- Use read(), readline(), readlines() to read data.
- Use write() and append() to write data.
- Use with statement to automatically close files.
- Handle exceptions using try-except for file operations.
- File handling is widely used in real-world applications like logs, reports, configuration files, etc.

Rahul Yadav

16. Exception Handling in Python

1 What is Exception Handling ?

- Exception handling is a way to handle runtime errors in Python.
- It helps the program to continue running even if an error occurs.
- We use try, except, else, finally and raise keywords.
- It makes our code more robust and reliable.
- Common exceptions: ValueError, TypeError, ZeroDivisionError, FileNotFoundError, etc.

2 Basic try - except Block

- We can use try block to write code that may raise an exception.
- If an exception occurs, it is caught by except block.
- We can handle different types of exceptions.

# Example	Output
try: a = 10 / 0 except ZeroDivisionError: print("Cannot divide by zero!")	Cannot divide by zero!

3 Handling Multiple Exceptions

- We can use multiple except blocks to handle different exceptions.
- Each except block handles a specific error.

# Example	Output
try: num = int("abc") except ValueError: print("Invalid number!") except TypeError: print("Type error!")	Invalid number!

4 try - else Block

- The else block runs when no exception occurs in the try block.
- It is useful to execute code when everything is successful.

# Example	Output

try:
  num = int("100")
except ValueError:
  print("Invalid number!")
else:
  print("Number is:", num)
	Number is: 100

5 try - finally Block

- The finally block always executes whether an exception occurs or not.
- It is useful for cleaning up resources like files, database connections, etc.

# Example	Output
try: f = open("data.txt", "r") content = f.read() print(content) except FileNotFoundError: print("File not found!") finally: print("This will always run!")	File not found! This will always run!

6 Raising an Exception

- We can use the raise keyword to manually raise an exception.
- It is useful for custom validation.

# Example	Output
age = -5 if age < 0: raise ValueError("Age cannot be negative!")	ValueError: Age cannot be negative!

7 Custom Exception Message

- We can provide our own error message while raising an exception.
- It helps in better debugging and understanding the issue.

# Example	Output
marks = 150 if marks > 100: raise ValueError("Marks cannot be greater than 100!")	ValueError: Marks cannot be greater than 100!

8 Common Built-in Exceptions

Exception	Description
ValueError	Raised when a function receives an invalid value.
TypeError	Raised when an operation is performed on an incompatible type.
ZeroDivisionError	Raised when dividing by zero.
FileNotFoundError	Raised when a file does not exist.
IndexError	Raised when a list index is out of range.
KeyError	Raised when a dictionary key is not found.

9 Using Exception with Functions

- We can handle exceptions inside a function.
- It makes the function more reliable and reusable.

# Example	Output

def divide(a, b):
  try:
  return a / b
  except ZeroDivisionError:
  return "Cannot divide by zero!"
print(divide(10, 0))
	Cannot divide by zero!

10 Key Takeaways

- Exception handling helps in writing robust code.
- Use try, except, else, finally and raise.
- Handle specific exceptions whenever possible.
- Use finally for cleanup operations.
- Raise custom exceptions for validations.
- Exception handling prevents program crashes.
- It improves code reliability and user experience.
- Widely used in real-world applications like file handling, API calls, database operations, etc.

Rahul Yadav

17. Object-Oriented Programming (OOP) in Python

1 What is OOP ?

- OOP is a programming paradigm based on the concept of "objects".
- It helps to organize code in a modular, reusable and maintainable way.
- Python supports OOP with classes, objects, inheritance, polymorphism, encapsulation and abstraction.
- It is widely used in real-world applications.

2 Class and Object

- A class is a blueprint or template to create objects.
- An object is an instance of a class.
- A class can have attributes (data) and methods (functions).

# Example	Output

`class Person:`
`  def __init__(self, name, age):`
`  self.name = name`
`  self.age = age`
`p = Person("Rahul", 28)`
`print(p.name)`
`print(p.age)`
	Rahul
	28

3 __init__() Constructor

- __init__() is a special method (constructor).
- It is automatically called when an object is created.
- It is used to initialize object attributes.

# Example	Output

`class Student:`
`  def __init__(self, name, marks):`
`  self.name = name`
`  self.marks = marks`
`s = Student("Preeti", 85)`
`print(s.name)`
`print(s.marks)`
	Preeti
	85

4 Instance Methods

- Instance methods are functions defined inside a class that operate on the instance variables (self).
- We can access instance variables using self keyword.

# Example	Output

`class Car:`
`  def __init__(self, brand):`
`  self.brand = brand`
`  def show(self):`
`  print("Brand:", self.brand)`
`c = Car("Toyota")`
`c.show()`
	Brand: Toyota

5 Class Variables vs Instance Variables

- Instance variables are unique for each object.
- Class variables are shared among all objects.

# Example	Output

`class Employee:`
`  company = "AU Bank"  # class variable`
`  def __init__(self, name):`
`  self.name = name  # instance variable`
`e1 = Employee("Rahul")`
`e2 = Employee("Preeti")`
`print(e1.company)`
`print(e2.company)`
	AU Bank
	AU Bank

6 Inheritance

- Inheritance allows a class to inherit properties and methods from another class.
- It helps in code reusability.

# Example	Output

`class Animal:`
`  def speak(self):`
`  print("Animal speaks")`
`class Dog(Animal):`
`  def bark(self):`
`  print("Dog barks")`
`d = Dog()`
`d.speak()`
`d.bark()`
	Animal speaks
	Dog barks

7 Method Overriding

- A child class can provide its own implementation of a method that is already defined in the parent class.
- This is called method ovorriding.

# Example	Output

`class Animal:`
`  def speak(self):`
`  print("Animal speaks")`
`class Cat(Animal):`
`  def speak(self):`
`  print("Cat meows")`
`c = Cat()`
`c.speak()`
	Cat meows

8 Encapsulation

- Encapsulation means hiding the internal details and protecting data from direct access.
- We can use private variables (__) to restrict access.

# Example	Output

`class BankAccount:`
`  def __init__(self, balance):`
`  self.__balance = balance  # private variable`
`  def get_balance(self):`
`  return self.__balance`
`acc = BankAccount(1000)`
`print(acc.get_balance())`
	1000

9 Polymorphism

- Polymorphism means the same method name can behave differently for different classes.
- It allows us to use a common interface for different types.

# Example	Output

`class Bird:`
`  def sound(self):`
`  print("Bird chirps")`
`class Duck:`
`  def sound(self):`
`  print("Duck quacks")`
`for obj in [Bird(), Duck()]:`
`  obj.sound()`
	Bird chirps
	Duck quacks

10 Key Takeaways

- OOP helps in writing clean, modular and maintainable code.
- Use classes and objects to model real-world entities.
- __init__() is used to initialize object attributes.
- Instance variables are unique, class variables are shared.
- Inheritance helps in code reusability.
- Method overriding allows custom behavior in child classes.
- Encapsulation protects data from direct access.
- Polymorphism allows the same method to behave differently.
- OOP is widely used in real-world applications like web, mobile, enterprise and AI systems.

Rahul Yadav

18. Iterators, Generators and Decorators in Python

1 What are Iterators ?

- An iterator is an object that allows us to iterate over a collection of data (like list, tuple, etc.) one element at a time.
- It implements two methods: __iter__() and __next__().
- We can create our own iterator using a class.
- Iterators are memory efficient because they return one element at a time.
- We can use iter() and next() functions to work with iterators.

2 Creating a Custom Iterator

- We can create a custom iterator by defining a class with __iter__() and __next__() methods.
- __iter__() returns the iterator object (self).
- __next__() returns the next value in the sequence.
- When there are no more items, it raises StopIteration.

# Example	Output

class CountIterator:
  def __init__(self, n):
  self.current = 1
  self.n = n
  def __iter__(self):
  return self
  def __next__(self):
  if self.current > self.n:
  raise StopIteration
  val = self.current
  self.current += 1
  return val
for num in CountIterator(5):
  print(num)
	1
	2
	3
	4
	5

3 Using iter() and next()

- We can convert any iterable (like list, tuple) into an iterator using iter().
- We can get the next element using next().
- When there are no more elements, StopIteration is raised.

# Example	Output
nums = [10, 20, 30] it = iter(nums) print(next(it)) print(next(it)) print(next(it))	10 20 30

4 What are Generators ?

- A generator is a special type of iterator.
- It is defined using a function with yield keyword.
- Generators are memory efficient as they generate values on the fly.
- They are useful when we have a large sequence of data.

5 Creating a Generator Function

- We can create a generator using the yield keyword.
- Each time the function is called, it returns a value and remembers the state for next time.

# Example	Output

def number_generator(n):
  for i in range(1, n + 1):
  yield i
gen = number_generator(5)
for num in gen:
  print(num)
	1
	2
	3
	4
	5

6 Generator Expression

- We can also create a generator using a compact syntax called generator expression.
- It is similar to list comprehension but uses parentheses ().
- It is more memory efficient than list comprehension.

# Example	Output
squares = (x * x for x in range(1, 6)) for num in squares: print(num)	1 4 9 16 25

7 What are Decorators ?

- A decorator is a function that takes another function and adds extra functionality to it without modifying its original code.
- Decorators use the @ symbol.
- They are widely used for logging, authentication, performance measurement, etc.

8 Creating a Simple Decorator

- A decorator is a function that takes a function as an argument and returns a new function.
- We use @decorator_name above the function.

# Example	Output

def my_decorator(func):
  def wrapper():
  print("Function is starting...")
  func()
  print("Function finished!")
  return wrapper
@my_decorator
def say_hello():
  print("Hello, Rahul!")
say_hello()
	Function is starting...
	Hello, Rahul!
	Function finished!

9 Decorator with Arguments

- We can also create decorators that accept arguments.

# Example	Output

def repeat(times):
  def decorator(func):
  def wrapper():
  for _ in range(times):
  func()
  return wrapper
  return decorator
@repeat(3)
def greet():
  print("Hello, Python!")
greet()
	Hello, Python!
	Hello, Python!
	Hello, Python!

10 Key Takeaways

- Iterators help us iterate over data one element at a time.
- iter() and next() are used to work with iterators.
- Generators use yield and are memory efficient.
- Generator expressions provide a compact way to create generators.
- Decorators add extra functionality to functions.
- Decorators are widely used in real-world applications like logging, authentication, caching, etc.

Rahul Yadav

19. Python Virtual Environments, pip and Project Structure

1 Why Virtual Environment ?

- A virtual environment is an isolated Python environment for a project.
- It keeps project dependencies separate from the global Python installation.
- Different projects can use different versions of libraries.
- It avoids version conflicts and makes the project more manageable.
- It is a best practice for Python development.

2 Creating a Virtual Environment

- We can use the venv module (built-in) to create a virtual environment.
- It creates a separate folder with its own Python interpreter and library packages.
- We can give any name to the environment folder.

#	Example (Command)	Description
	python -m venv myenv	Create virtual environment
	python3 -m venv myenv	For Linux/Mac
	python -m venv .venv	Common folder name
	dir	Check the created folder (Windows)
	ls	Check the created folder (Linux/Mac)

3 Activating the Virtual Environment

- After creation, we need to activate the environment to start using it.
- The command is different for different operating systems.

#	Command	Platform
	myenv\Scripts\activate	Windows (Command Prompt)
	myenv\Scripts\Activate.ps1	Windows (PowerShell)
	source myenv/bin/activate	Linux / Mac
	source .venv/bin/activate	Using .venv folder

* After activation, you will see the environment name in your terminal like (myenv) before the path.

4 Deactivating the Virtual Environment

- We can deactivate the virtual environment when we are done working.
- It will return to the global Python environment.

#	Command	Description
	deactivate	Deactivate virtual environment (Works in all OS)

5 Installing Packages using pip

- pip is a package manager used to install Python libraries.
- After activating the virtual environment, we can install any package using pip.

#	Example	Description
	pip install requests	Install a package
	pip install pandas numpy	Install multiple packages
	pip list	List installed packages
	pip show requests	Show package details
	pip uninstall requests	Uninstall a package
	pip freeze > requirements.txt	Save installed packages
	pip install -r requirements.txt	Install packages from file

6 Checking Python and pip Version

- We can check the Python and pip version to confirm our environment.
- This helps to ensure we are using the correct Python and pip inside the virtual environment.

#	Command	Description
	python --version	Check Python version
	pip --version	Check pip version
	which python	Show Python path (Linux/Mac)
	where python	Show Python path (Windows)

7 Requirements File

- We can create a requirements.txt file to save all project dependencies.
- It helps to easily share and install the same environment on another machine.

#	Example	Description
	pip freeze > requirements.txt	Generate requirements file
	pip install -r requirements.txt	Install from requirements file

8 Basic Project Structure

- A well-organized project structure helps in maintainability and scalability.
- We can create folders for source code, modules, tests, and configuration files.

```
my_project/
├── .venv/              # virtual environment
├── src/                # source code
│   ├── main.py         # main file
│   └── utils.py        # utility functions
├── tests/              # test files
├── requirements.txt    # project dependencies
└── README.md           # project description
```

9 Using Virtual Environment in VS Code

- Open the project folder in VS Code.
- Open Command Palette (Ctrl + Shift + P).
- Search for "Python: Select Interpreter".
- Choose the interpreter from the .venv folder.
- Now VS Code will use the virtual environment for running and debugging.

10 Key Takeaways

- Virtual environment keeps dependencies isolated.
- We use venv module to create a virtual environment.
- Activate the environment before installing packages.
- pip is used to manage packages.
- Use requirements.txt to share dependencies.
- Maintain a clean project structure for better code organization.
- This setup is widely used in real-world Python projects.

Rahul Yadav

20. Python Best Practices, Useful Tools and Final Project

1 Python Best Practices

- Write clean, readable and maintainable code.
- Use meaningful variable and function names.
- Follow PEP 8 coding style guide.
- Keep functions small and focused (Single Responsibility).
- Use comments and docstrings.
- Handle exceptions properly.
- Avoid global variables.
- Use virtual environments for project isolation.
- Use type hints for better code understanding.
- Write modular code and organize it in packages.

2 PEP 8 Naming Conventions

Element	Style	Example
Variable / Function	snake_case	user_name
Class	PascalCase	Employee
Constant	UPPER_CASE	MAX_LIMIT
Module / File	snake_case	user_service.py
Package	snake_case	my_package
Private variable	_single_leading	_name
Very private	__double_leading	__value
Method (internal)	_single_leading	_calculate()

3 Using Docstrings

- Docstrings are used to document modules, classes, functions and methods.
- They help other developers understand the purpose, parameters and return values.
- We can follow the Google or Numpy style for docstrings.

# Example	Output (Help)
def add(a: int, b: int) -> int: """ Add two numbers. Args: a (int): First number b (int): Second number Returns: int: Sum of two numbers """ return a + b	help(add) Help on function add: add(a: int, b: int) -> int Add two numbers. Args: a (int): First number b (int): Second number Returns: int: Sum of two numbers

4 Useful Python Tools

Tool	Purpose
pip	Install and manage packages
venv / virtualenv	Create virtual environments
VS Code / PyCharm	Popular IDEs for Python
black	Auto code formatter (PEP 8)
flake8	Linting tool to check code style
mypy	Static type checker
pytest	Testing framework
jupyter	Interactive notebook for data analysis
pandas	Data analysis library
numpy	Numerical computing library
matplotlib	Data visualization library
requests	Make HTTP requests
fastapi	Build modern web APIs
docker	Containerize Python applications

5 Code Formatting with black

- black automatically formats your Python code.
- It follows PEP 8 guidelines.
- It helps maintain consistent code style in a team.

# Example	Description
pip install black black app.py	Install black formatter Format the code in app.py file

6 Testing with pytest

- pytest is a popular testing framework.
- It is simple and easy to use.
- We can write test cases to ensure our code works correctly.

# Example	Description

def add(a, b):
  return a + b
 	
def test_add():
  assert add(2, 3) == 5
	Run using:
	pytest test_file.py
	
	Checks if add()
	function works correctly.

7 Error Handling Best Practices

- Always handle expected exceptions.
- Use meaningful error messages.
- Log errors for debugging.
- Do not silently ignore exceptions.

# Example	Description
try: x = int(input("Enter a number: ")) print(10 / x) except ValueError: print("Please enter a valid number!") except ZeroDivisionError: print("Cannot divide by zero!")	Handle multiple exceptions with meaningful messages.

8 Final Project - Simple Web API

- We will create a simple REST API using FastAPI.
- The API will manage a list of users.
- We will use best practices like modular code, virtual environment and proper project structure.

Project Structure
my-api/ ├── app/ │ ├── __init__.py │ ├── main.py │ ├── models.py │ ├── routes.py │ └── utils.py ├── tests/ ├── requirements.txt └── README.md

main.py (Example)
from fastapi import FastAPI app = FastAPI() @app.get("/") def home():   return {"message": "Hello, Rahul!"}

9 Running the Project

- Create a virtual environment.
- Install dependencies.
- Run the FastAPI application.

# Commands	Description
python -m venv .venv	Create virtual environment
source .venv/bin/activate	Activate (Linux/Mac)
.venv\Scripts\activate	Activate (Windows)
pip install -r requirements.txt	Install dependencies
uvicorn app.main:app --reload	Run the FastAPI server

10 Key Takeaways

- Follow best practices to write clean and maintainable code.
- Use tools like black, flake8 and pytest.
- Use virtual environment to manage dependencies.
- Write proper docstrings and handle exceptions.
- Organize your project structure.
- Build real-world projects to apply your knowledge.
- These practices will help you become a professional Python developer and prepare you for industry-level development.

Rahul Yadav